



UNIVERSITY OF PETROLEUM AND ENERGY
STUDIES
DEHRADUN

Applied DevOps

Experiment 10

MTECH - School of Computer Science
CYBER SECURITY AND FORENSICS

Submitted To: Dr. KrishnaRaj

Name: Jigesh Sheoran

SAP ID: 590025428

Objective — To design and implement a monitoring and incident management system using Docker Compose with Prometheus, Grafana, and Alertmanager to ensure application reliability and observability.

Task to be Performed

Task 1 : Web App + Containerization

Step 1: Create project

```
[sheoraninfosec@Jigeshs-MacBook-Air ~ % mkdir exp10 && cd exp10  
sheoraninfosec@Jigeshs-MacBook-Air exp10 % nano app.py
```

Step 2: create app

```
UW PICO 5.09 File: app.py Modified  
  
from flask import Flask  
app = Flask(__name__)  
  
@app.route("/")  
def home():  
    return "App is running successfully!"  
  
@app.route("/error")  
def error():  
    return "Error occurred!", 500  
  
app.run(host="0.0.0.0", port=5000)
```

Step 3: Dockerfile

```
UW PICO 5.09 File: Dockerfile  
  
FROM python:3.9  
WORKDIR /app  
COPY . .  
RUN pip install flask  
CMD ["python", "app.py"]
```

Task 2: docker-compose.yml

```
[sheoraninfosec@Jigeshs-MacBook-Air exp10 % nano Dockerfile  
sheoraninfosec@Jigeshs-MacBook-Air exp10 % nano docker-compose.yml
```

```
exp10 — nano docker-compose.yml — 80x30
UW PICO 5.09 File: docker-compose.yml Modified
version: '3'

services:
  web:
    build: .
    ports:
      - "5000:5000"

  prometheus:
    image: prom/prometheus
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml
    ports:
      - "9090:9090"

  grafana:
    image: grafana/grafana
    ports:
      - "3000:3000"

  node-exporter:
    image: prom/node-exporter
    ports:
      - "9100:9100"

^G Get Help ^O WriteOut ^R Read File ^Y Prev Pg ^K Cut Text ^C Cur Pos
^X Exit ^J Justify ^W Where is ^V Next Pg ^U UnCut Text ^T To Spell
```

Prometheus config

```
UW PICO 5.09 File: prometheus.yml Modified
global:
  scrape_interval: 5s

scrape_configs:
  - job_name: 'node'
    static_configs:
      - targets: ['node-exporter:9100']
```

```
[sheoraninfosec@Jigeshs-MacBook-Air exp10 % docker compose up -d]
WARN[0000] /Users/sheoraninfosec/exp10/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] up 18/26
  ? Image prom/prometheus [#####] 2.099MB / 140.1MB Pulling 12.0s
  ✓ Image prom/node-exporter Pulled 11.7s
  ? Image grafana/grafana [#####] 10.49MB / 323.3MB Pulling 12.0s
```

Task 3: Grafana Dashboard

localhost:3000/login



Welcome to Grafana

Email or username

email or username

Password

password

Log in

[Forgot your password?](#)

 **prometheus**

Type
Prometheus

Alerting
Supported

[Explore data](#)

[Build a dashboard](#)

Type: Prometheus

[Settings](#)

[Permissions](#)

[Insights](#)

[Cache](#)



Configure your Prometheus data source below

Or skip the effort and get Prometheus as fully-managed, scalable, and hosted data source from Grafana Labs with the [free-forever Grafana Cloud plan](#).

Name 

prometheus

Default 

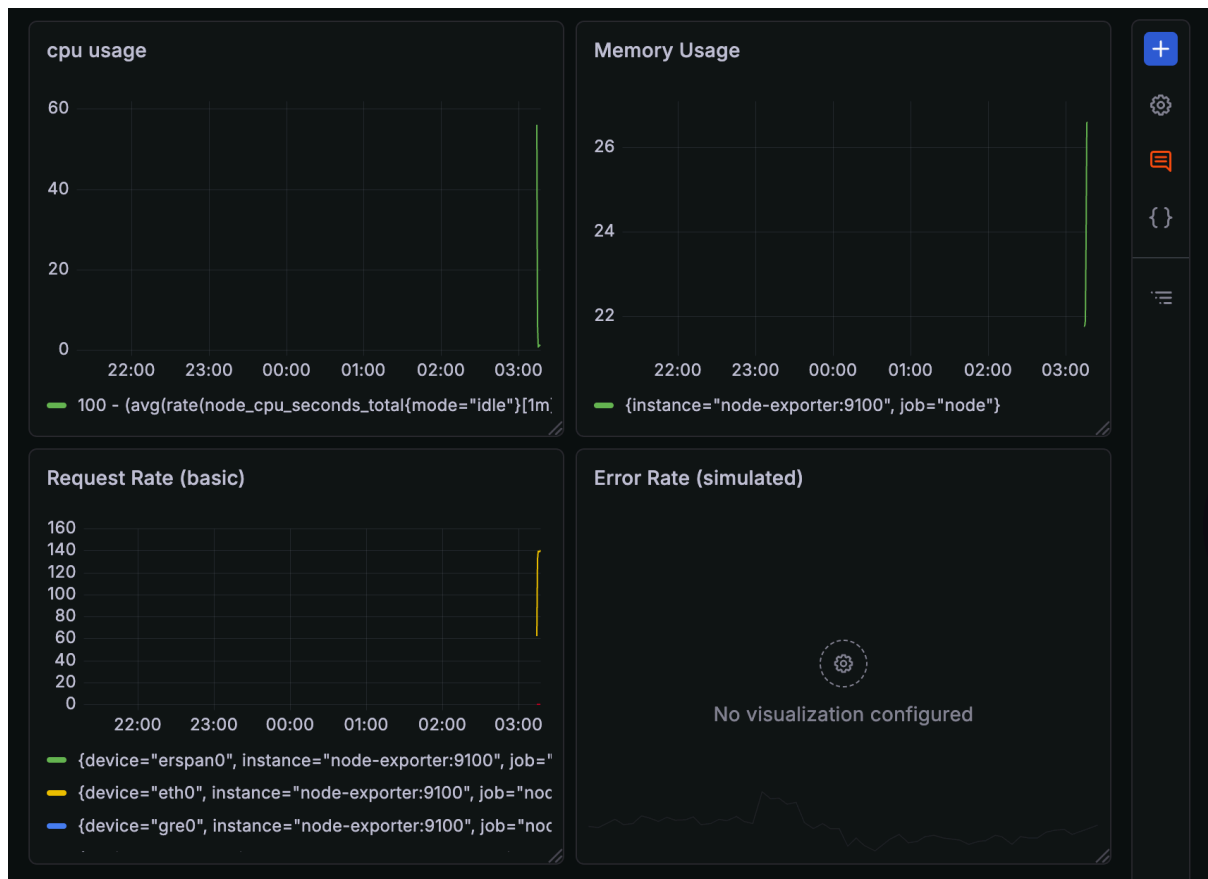
Before you can use the Prometheus data source, you must configure it below or in the config file. For detailed instructions, [view the documentation](#).

Fields marked with * are required

Connection

Prometheus server URL *

[Info](#)



Task 4:

Service Level Indicator (SLI)

A Service Level Indicator (SLI) is a quantitative measure used to evaluate the performance of a system.

It represents how well the service is performing.

Formula:

$$\text{SLI} = \frac{\text{Number of Successful Requests}}{\text{Total Requests}}$$

Example: If the number of successful requests is 950 out of 1000 total requests:

$$\text{SLI} = 950 / 1000 = 0.95 = 95\%$$

Service Level Objective (SLO)

A Service Level Objective (SLO) is the target value set for a service's performance. It defines the expected level of reliability.

Example SLOs:

- Availability SLO: 99%
- Error Rate SLO: Less than 1%

Interpretation:

- If $SLI \geq SLO$: System performance is acceptable
- If $SLI < SLO$: Improvement is required

Availability Percentage

Availability measures the proportion of time the system is operational and accessible.

Formula:

$$\text{Availability (\%)} = (\text{Uptime} / \text{Total Time}) \times 100$$

Error Rate

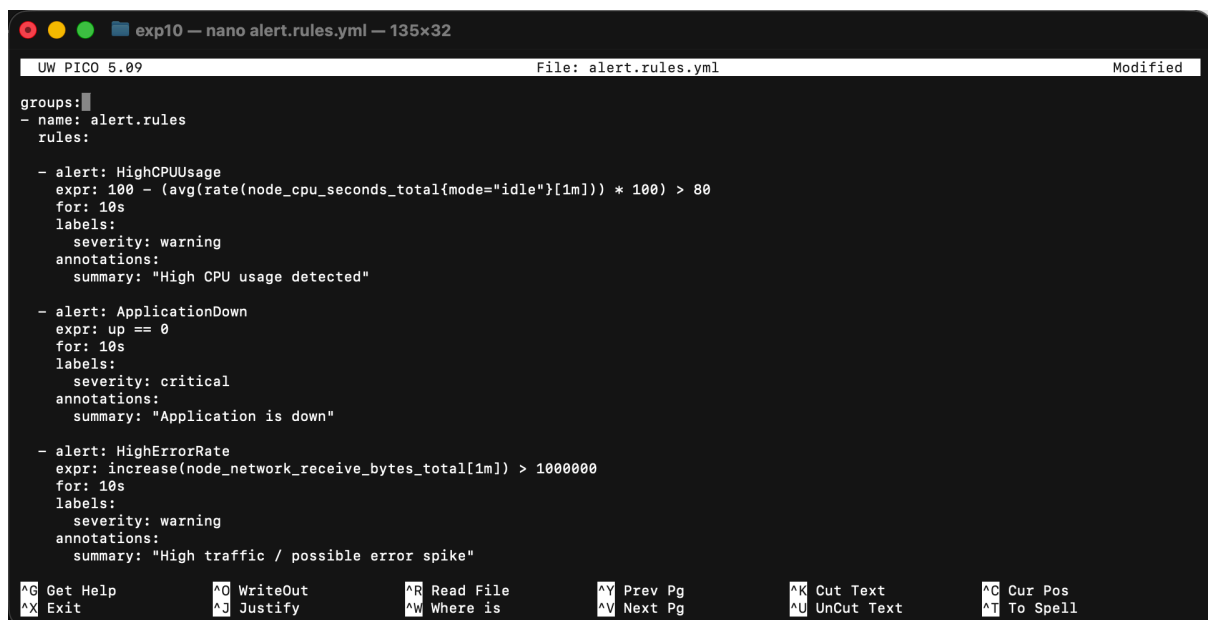
Error rate is the percentage of requests that result in failures.

Formula:

$$\text{Error Rate (\%)} = (\text{Number of Failed Requests} / \text{Total Requests}) \times 100$$

Task 5: Configuration Alerts

```
sheoraninfosec@Jigeshs-MacBook-Air exp10 % nano alert.rules.yml
```



```
exp10 — nano alert.rules.yml — 135x32
UW PICO 5.09 File: alert.rules.yml Modified
groups:
- name: alert.rules
  rules:
    - alert: HighCPUUsage
      expr: 100 - (avg(rate(node_cpu_seconds_total{mode="idle"}[1m])) * 100) > 80
      for: 10s
      labels:
        severity: warning
      annotations:
        summary: "High CPU usage detected"
    - alert: ApplicationDown
      expr: up == 0
      for: 10s
      labels:
        severity: critical
      annotations:
        summary: "Application is down"
    - alert: HighErrorRate
      expr: increase(node_network_receive_bytes_total[1m]) > 1000000
      for: 10s
      labels:
        severity: warning
      annotations:
        summary: "High traffic / possible error spike"
^G Get Help      ^O WriteOut      ^R Read File     ^V Prev Pg      ^K Cut Text      ^C Cur Pos
^X Exit          ^J Justify       ^W Where is      ^V Next Pg      ^U UnCut Text    ^T To Spell
```

```
UW PICO 5.09 File: prometheus.yml

global:
  scrape_interval: 5s

scrape_configs:
  - job_name: 'node'
    static_configs:
      - targets: ['node-exporter:9100']

rule_files:
  - "alert.rules.yml"

alerting:
  alertmanagers:
    - static_configs:
      - targets:
        - alertmanager:9093
```

Add this service to nano docker-compose.yml

```
alertmanager:
  image: prom/alertmanager
  ports:
    - "9093:9093"
```

Restart everything:

```
exp10 — docker-compose • docker compose up -d — 135x32

[sheoraninfosec@Jigeshs-MacBook-Air exp10 % clear]
[sheoraninfosec@Jigeshs-MacBook-Air exp10 % nano alert.rules.yml]
[sheoraninfosec@Jigeshs-MacBook-Air exp10 % nano prometheus.yml]
[sheoraninfosec@Jigeshs-MacBook-Air exp10 % nano docker-compose.yml]
[sheoraninfosec@Jigeshs-MacBook-Air exp10 % docker compose down]
WARN[0000] /Users/sheoraninfosec/exp10/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to
avoid potential confusion
[+] down 5/5
✓ Container exp10-web-1 Removed 0.0s
✓ Container exp10-prometheus-1 Removed 0.5s
✓ Container exp10-grafana-1 Removed 0.4s
✓ Container exp10-node-exporter-1 Removed 0.4s
✓ Network exp10_default Removed 0.2s
[sheoraninfosec@Jigeshs-MacBook-Air exp10 % docker compose up -d]
WARN[0000] /Users/sheoraninfosec/exp10/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to
avoid potential confusion
[+] up 0/1
! Image prom/alertmanager Pulling 2.8s
```

Alertmanager Alerts Silences Status Settings Documentation [New Silence](#)

Filter Group

Receiver: All ☐ Silenced ☐ Inhibited ☐ Muted

+ [Silence](#)

Custom matcher, e.g. `env="production"`

+ Expand all groups

No alert groups found

Conclusion

This experiment compared container-based environments and virtual machines in terms of isolation, performance, and automation. Containers provided fast, lightweight deployment with process-level isolation, while virtual machines offered stronger OS-level isolation but required more resources and setup time.

The results showed that containers are more efficient and suitable for modern DevOps and scalable systems, whereas virtual machines are better suited for scenarios requiring complete system isolation and higher security.

-----END OF DOCUMENT -----